

SESSION LIVE • 60 MIN • ZÉRO CLOUD

LLM & développement agentique

Tout ça tourne chez vous — à condition de savoir où passe la mémoire.

m-max • octobre 2026 • un laptop CPU, un serveur sur le LAN, aucun cloud

Le programme : exécuter, orchestrer, explorer

ACTE 1

Exécuter

Ce qui se passe entre Entrée et le premier mot : mémoire, quantification, prefill, decode, KV cache.

LIVE : de Hugging Face à la ligne de log

ACTE 2

Orchestrer

Ce qu'un agent envoie au modèle à chaque tour, et pourquoi ça marche : outils, boucle, contexte, cache.

LIVE : un agent local corrige un bug • une skill en direct

ACTE 3

Explorer

Le paysage à l'automne 2026 : où en sont les modèles open-weights, et sur quel matériel les faire tourner.

Les chiffres du jour, sourcés

Deux fils rouges : la mémoire est la monnaie • rien ne sort de la salle.

ACTE 1 • EXÉCUTER

Entre Entrée et le premier mot



Exécuter

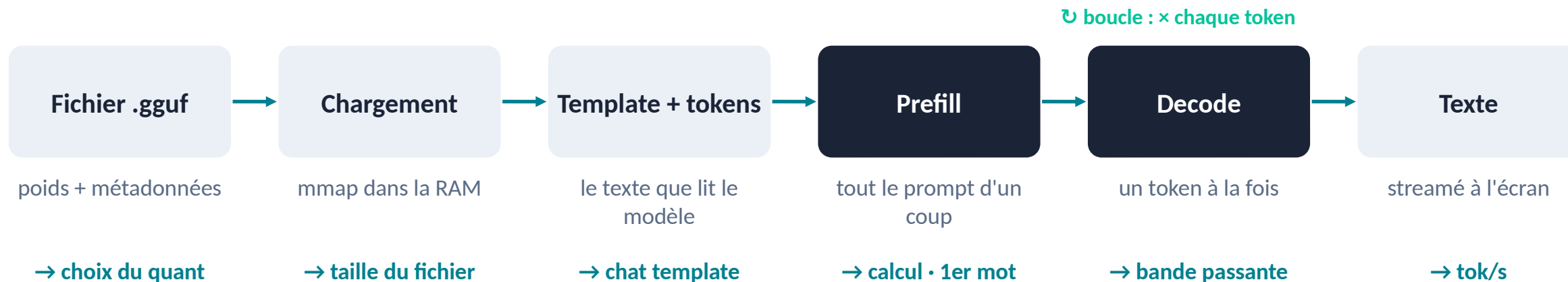


Orchestrer



Explorer

Six étapes entre Entrée et le premier mot



Deux ressources, deux limites

La RAM : il faut y loger les poids et le KV cache.

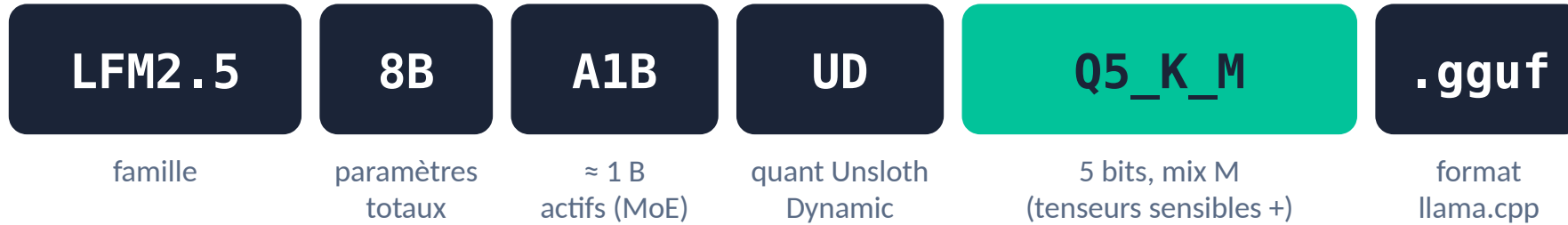
La bande passante mémoire : le decode relit les poids à chaque token.

Le calcul (CPU/GPU) ne limite vraiment que le prefill.

KV cache : rempli par le prefill, relu à chaque token

Chaque étape a un réglage et une mesure. **On les prend une par une.**

Un fichier GGUF se lit comme une étiquette



LFM2.5-8B-A1B-UD-Q5_K_M.gguf · 5,92 GiB sur le disque

Dedans : les poids

Des milliards de nombres, rangés en tenseurs, déjà quantifiés.

Un seul fichier, pensé pour être mappé en mémoire tel quel.

Taille du fichier ≈ RAM occupée par les poids.

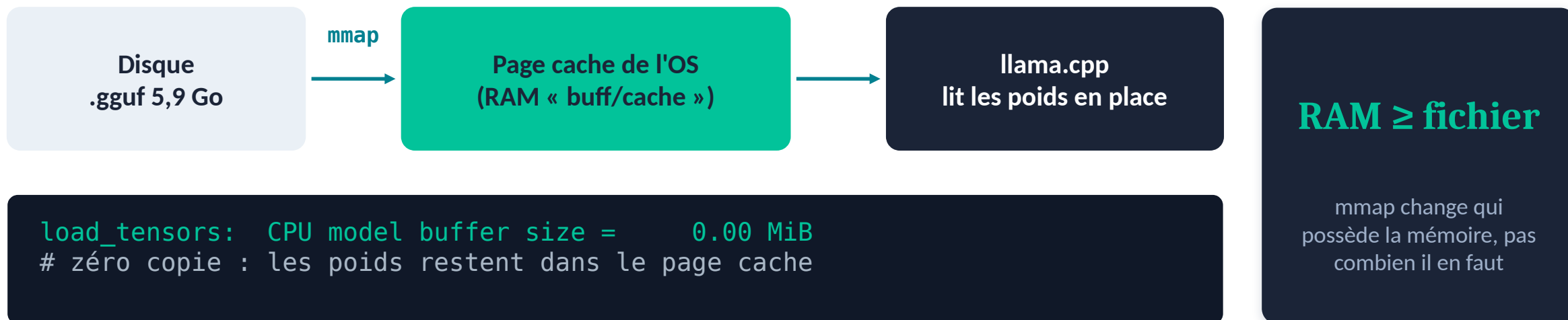
... et des métadonnées

Architecture, nombre de couches, contexte d'entraînement.

Le chat template (Jinja) : comment formater une conversation.

D'où « rien à configurer » : -c 0 lit le contexte du modèle.

Charger un modèle : il est mappé, pas copié



Ce que ça change en pratique

1er lancement lent (lecture disque), 2e instantané : le fichier est encore en cache.

free / top rangent les poids en « buff/cache », pas en « utilisé » : la RAM est bien prise.

Si le fichier dépasse la RAM : relu depuis le disque à chaque token → des minutes par token.

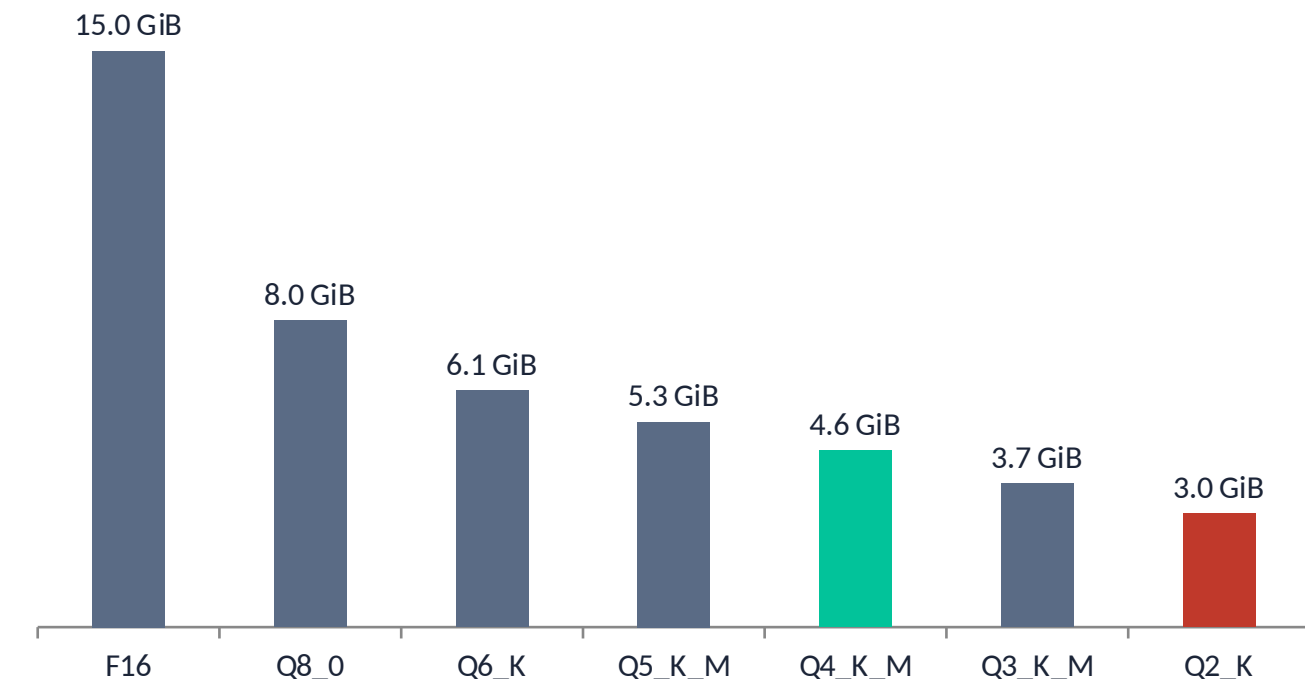
Réglage

-lm / --load-mode

(auto, mmap, mlock...)

remplace --no-mmap et --mlock

Quantifier : 15 GiB → 4,6 GiB, presque la même tête



Écart à l'original (KLD) : Q8_0 0,001 · Q6_K 0,006 · Q5_K_M 0,011 · Q4_K_M 0,028 · Q3_K_M 0,084 · Q2_K 0,445

Lire le graphique

Les poids sont stockés par blocs, avec un facteur d'échelle par bloc : moins de bits par poids.

Go ≈ milliards de paramètres × bits par poids ÷ 8

Q4_K_M ≈ 4,9 bits → ~0,6 Go par milliard.

Q4_K_M est le coude : ÷3,3 en taille pour un écart minime. Sous 3 bits, la qualité s'effondre.

Un gros modèle quantifié bat un petit modèle entier : **13B en Q4_K_M (7,3 Go) > 7B en F16 (13 Go).**

Prefill lit, decode écrit — pas au même prix

Prefill • le prompt d'un coup

Tous les tokens du prompt sont traités en parallèle et remplissent le KV cache.

Limité par le calcul. Mesure : temps avant le 1er mot (TTFT).

Decode • un token à la fois

Pour chaque token : relire tous les poids actifs, produire un score par token possible, tirer au sort.

Limité par la bande passante mémoire. Mesure : tok/s.

$$\text{tok/s en decode} \lesssim \text{bande passante mémoire} \div \text{octets des poids actifs}$$

Mesuré	Prefill	Decode	Rapport
Laptop CPU (LFM2.5-8B-A1B)	98 t/s	26 – 35 t/s	× 2 à 4
Serveur LAN (Qwen3.8-Flash-Next)	912 t/s	36 t/s	× 25

Mesuré sur ce laptop : le decode lit la mémoire



Même bande passante effective

Dense : $2,4 \text{ Go relus} \times 14,8 \text{ t/s} \approx 38 \text{ Go/s}$

MoE : $\sim 1,1 \text{ Go actifs} \times 34,6 \text{ t/s} \approx 39 \text{ Go/s}$

Le CPU ne calcule presque pas : il attend la mémoire.

Plus de threads ≠ plus vite

6 threads (P-cores) → 34,6 t/s

14 threads (+ E-cores) → 26,4 t/s : -24 %

Fichier 2,4× plus gros, decode 2,3× plus rapide : **seuls les poids actifs sont relus.**

Le KV cache : la mémoire de la conversation

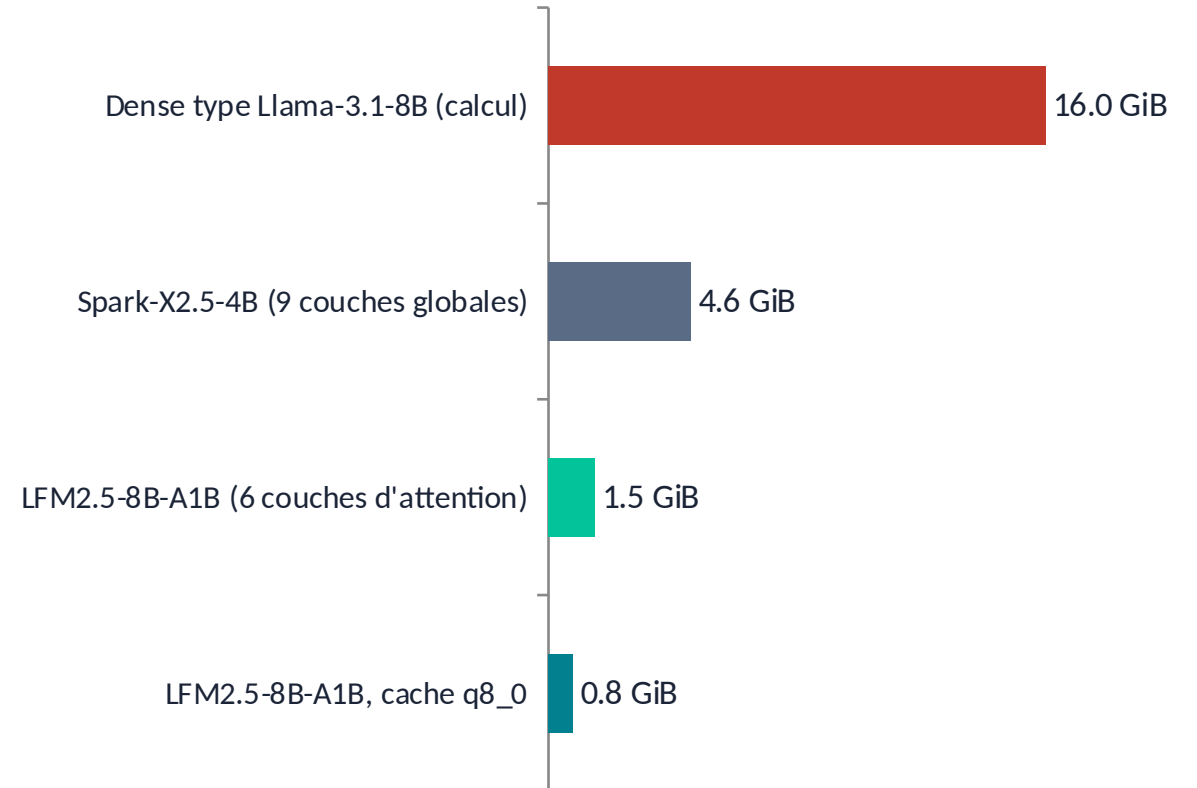
Ce que c'est

Pour chaque token déjà lu, chaque couche d'attention garde ses clés et valeurs (K, V) : on ne recalcule pas le passé à chaque nouveau token.

Réservé en entier au démarrage, pour tout -c.

$2 \times \text{couches} \times \text{têtes KV} \times \text{dim tête} \times \text{octets} \times \text{tokens}$

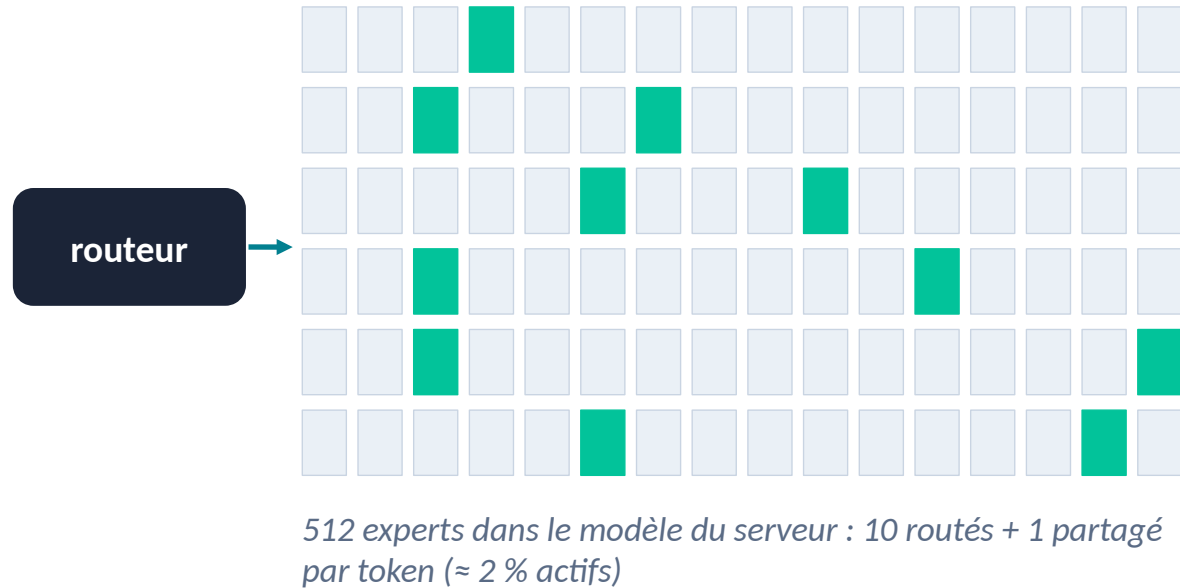
Llama-3.1-8B : $2 \times 32 \times 8 \times 128 \times 2 \times o$
= 128 Kio/token → 16 Gio à 128k



Le contexte se paie en RAM, et l'architecture change tout : de 1,5 à 16 GiB pour la même longueur.

MoE : la RAM paie le total, la vitesse paie l'actif

Pour chaque token, un routeur choisit quelques experts :



177 B

paramètres au total → 111
Go de RAM

6 B

actifs par token → ~36 tok/s

Bonus : décodage spéculatif (MTP)

Le modèle propose plusieurs tokens d'avance et les vérifie en une passe : 61 % acceptés, ~2,8 tokens par passe sur le serveur.

Le secret du local en 2026 : **de gros modèles MoE avec peu de paramètres actifs, et beaucoup de RAM.**

Dimensionner avant de télécharger

RAM $\approx$ poids + KV cache + $\sim 0,3$ Go

fichier .gguf · contexte réservé (-c) · tampons de calcul

tok/s $\approx$ bande passante $\div$ poids actifs

Go/s de la mémoire · octets relus par token

Ce laptop · LFM2.5-8B-A1B Q5_K_M
RAM = 5,9 + 0,4 (32k) + 0,1
 $\approx 6,4$ Go ✓ **sur 30 Go**
tok/s ≈ 38 Go/s $\div \sim 1,1$ Go
 ≈ 35 t/s ✓ **mesuré : 34,6**

Machine	Ce qui y tourne bien
Laptop 16-32 Go, CPU	Chat, complétion : MoE de 4-8 B, ~ 30 t/s
64-128 Go, mémoire rapide	Agent de code : MoE de 30 à 120 B
Serveur de la démo	MoE 177 B (6 B actifs), 111 Go

Réponse au sondage : un 8B en Q4_K_M ≈ 5 Go de poids — il tient sur votre laptop. S'il est MoE, il y est même rapide.

LIVE

De Hugging Face à la ligne de log

1 Lire une page de modèle

unsloth/LFM2.5-8B-A1B-GGUF : les quants et leurs tailles, avec la règle $Go \approx B \times \text{bits} \div 8$.

2 hf download, 397 Mo

Qwen3-0.6B en Q4_K_M, téléchargé pendant la phrase.

3 llama-cli : lire le log

KV cache 4 480 MiB pour 373 Mo de poids. Puis -c 4096 : ~450 MiB.

```
$ hf download unsloth/Qwen3-0.6B-GGUF \
  --include "Qwen3-0.6B-Q4_K_M.gguf" \
  --local-dir /models/Qwen3-0.6B
$ llama-cli -m .../Qwen3-0.6B-Q4_K_M.gguf \
  -p Bonjour -st -v 2>&1 \
  | grep -E "kv_cache|Prompt:"
llama_kv_cache: size = 4480.00 MiB
[ Prompt: 330.7 t/s | Generation: 97.3 t/s ]
```

× 12 : la mémoire de la conversation pèse douze fois le modèle.



Exécuter



Orchestrer



Explorer

ACTE 2 • ORCHESTRER

Le harnais et sa conversation avec le LLM



Exécuter



Orchestrer



Explorer

Le modèle est une fonction sans état

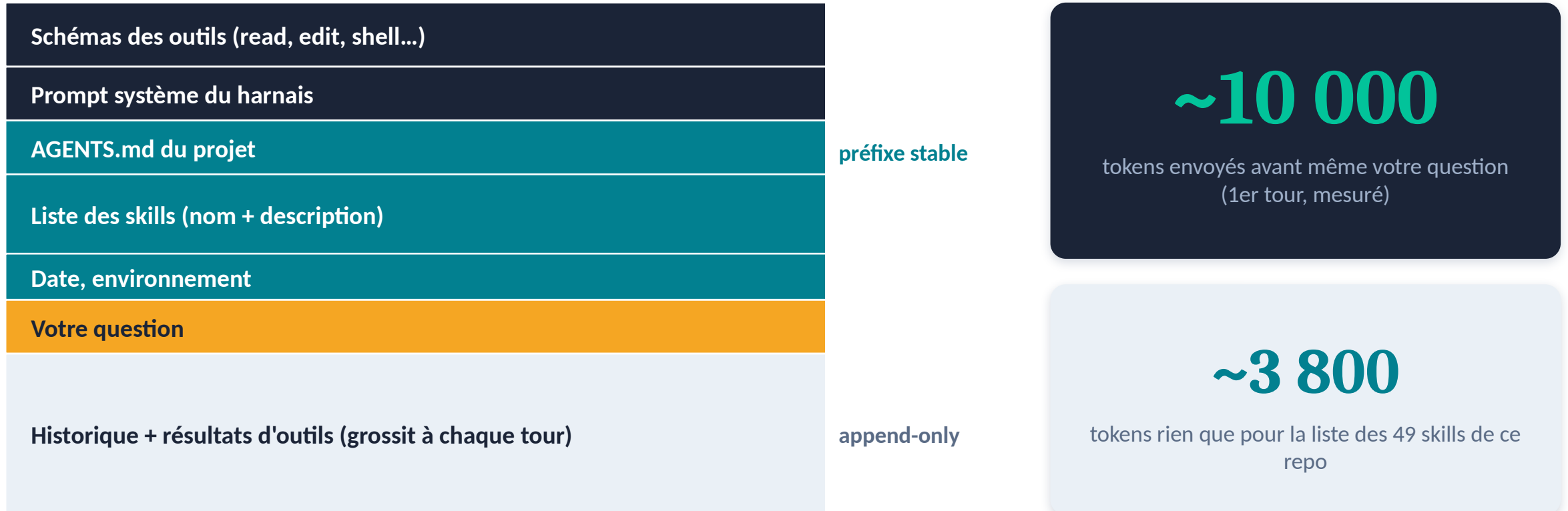


Il ne garde rien d'une requête à l'autre. Tout ce qui ressemble à de la mémoire, des outils ou de l'autonomie est fait par le programme autour : le harnais.

Qui	Fait quoi
Modèle	choisit l'outil et les arguments (du texte)
Template Jinja	traduit JSON ⇔ texte balisé
llama-server	parse les appels, KV cache
Harnais	prompts, exécution, contexte
Vous	AGENTS.md, skills, permissions

Harnais = boucle + outils + mémoire + permissions, autour d'un modèle qui ne fait que compléter du texte.

Anatomie d'une requête d'agent



Chaque ligne d'AGENTS.md, chaque skill installée, chaque outil MCP **est payé à chaque tour.**

Le tool calling, c'est du texte et un parser

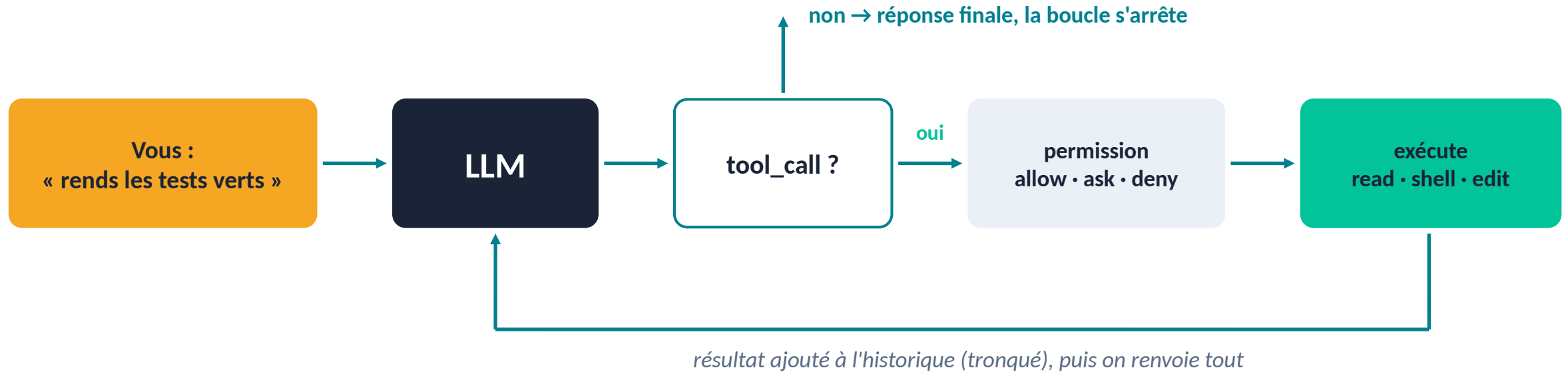
```
<|im_start|>system
# Tools
<tools>{"name": "read", "parameters": ...}</tools>
...You are an AI agent running in OpenCode...
<|im_start|>user
Corrige le test qui échoue dans calc.py
<|im_start|>assistant
<think>Je dois d'abord lire calc.py.</think>
<tool_call>
<function=read>
<parameter=path>calc.py</parameter>
</function></tool_call>
<|im_start|>user
<tool_response>
2:         return a - b
</tool_response>
```

- 1 Le harnais envoie tools (schémas JSON)
- 2 Le template les écrit dans le prompt
- 3 Le modèle génère <tool_call>...
- 4 llama-server parse → tool_calls JSON
- 5 Le harnais exécute l'outil
- 6 Le résultat revient en <tool_response>

Grammaire « lazy » : dès <tool_call>, la sortie est contrainte au schéma.

Template inconnu → mode Generic, moins fiable.

La boucle — et quand elle s'arrête



~8

appels d'outils par réponse (855 tool-calls / 101 stop, mesuré)

Ce qu'il faut savoir

Elle s'arrête quand le modèle répond sans appel d'outil — pas de limite d'étapes par défaut.
Un modèle qui rate ses appels d'outils tourne en rond ou s'arrête trop tôt : c'est un critère de choix.

La boucle n'est pas dans le modèle : c'est le harnais qui la fait tourner.

Le KV cache, deuxième acte : le cache de prompt

Le début du prompt est identique d'un tour à l'autre : llama-server garde son KV cache et ne calcule que la fin.

94,6 %

des tokens de prompt servis par le cache du serveur

20 s → 1 s

avant le 1er mot : 1er tour d'une session vs tours suivants

4 min

de prefill au lieu de ~81 min sans le cache

Sur le laptop CPU

~10 000 tokens à ~100 t/s de prefill = ~100 s avant le 1er mot, à chaque nouvelle session.

D'où le serveur : le chat tient sur un laptop, l'agent veut une machine rapide.

Ne touchez pas au début du prompt

OpenCode ordonne ses blocs du plus stable au plus variable, exprès.

Ajouter un serveur MCP en cours de session invalide tout le cache.

Brancher OpenCode : une base URL

```
"providers": {  
  "llama.cpp": {  
    "package": "aisdk:@ai-sdk/openai-compatible",  
    "settings": {  
      "baseUrl": "http://192.168.0.110:8080/v1"  
    },  
    "models": {  
      "Qwen3.8-Flash-Next": {  
        "capabilities": { "tools": true, ... },  
        "limit": { "context": 262144 }  
      }  
    }  
  }  
}
```

Laptop
OpenCode



llama-server
192.168.0.110

API compatible OpenAI, sur le LAN

Trois points d'attention

capabilities.tools : sinon le modèle n'a pas d'outils.

limit.context : la vraie fenêtre, pour la compaction.

Test de vie : curl .../v1/models

Le modèle et le harnais ne se connaissent que par cette URL.

LIVE

OpenCode corrige un bug, 100 % local

1 Le constat

python3 -m unittest dans demo/ : 9 tests, 3 rouges. Un bug de facturation. Je ne dis pas où.

2 L'agent travaille

read → shell → read → edit → shell. Et le log du serveur montre le cache au travail.

3 git diff

Une seule valeur change. « Il n'a pas deviné : il a mesuré. »

```
$ curl -s 192.168.0.110:8080/metrics
prompt_tokens_total      239 162
prompt_tokens_cached_total 4 212 240
predicted_tokens_seconds 35.9
```

4,4 M tokens de prompt servis par ce serveur pendant la préparation du talk, **94,6 % depuis le cache. Rien n'a quitté le LAN.**



Exécuter



Orchestrer



Explorer

Étendre l'agent sans coder

AGENTS .md

Les règles du projet

Un fichier markdown à la racine, injecté dans chaque requête. Court et précis : chaque ligne est payée à chaque tour.

SKILL .md

Savoir-faire à la demande

Divulgaration progressive : nom + description au départ, le corps quand la tâche s'y prête, les fichiers annexes ensuite.

MCP

Des outils externes

Un protocole client/serveur : tickets, base de données, navigateur... OpenCode v2 les expose via un catalogue texte + un outil exécute.

Chaque extension a un coût en contexte : **même « progressives », 49 descriptions de skills pèsent ~3 800 tokens à chaque session.**

LIVE

Écrire une skill en direct

1 Un dossier, un fichier

.opencode/skills/explain-file/SKILL.md, frontmatter name + description.

2 Quatre consignes

Rôle · Structure · Subtilités · Limites. Quinze lignes en tout.

3 L'invocation

« explique demo/pricing.py » : l'agent voit la skill, la charge, l'applique.

```
---  
name: explain-file  
description: Explique un fichier  
             de code (rôle, structure,  
             subtilités, limites)  
---  
1. Lis le fichier en entier.  
2. Rends : Rôle, Structure,  
           Subtilités, Limites.
```



Exécuter



Orchestrer



Explorer

ACTE 3 • EXPLORER

Le paysage, automne 2026



Exécuter

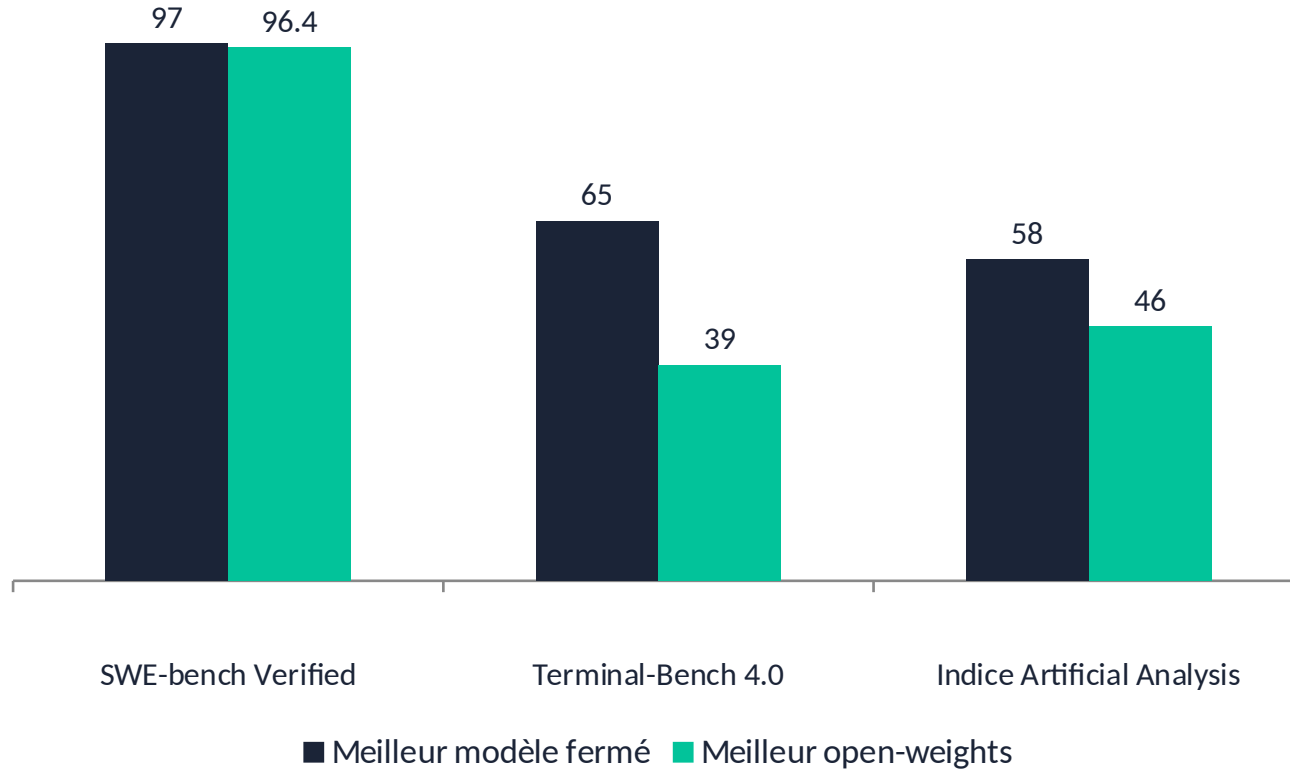


Orchestrer



Explorer

Open-weights : l'écart s'est fermé... sur les tâches courtes



Ce qu'il faut retenir

MoE partout : 1,5 à 6 B actifs pour 8 à 180 B et plus au total.

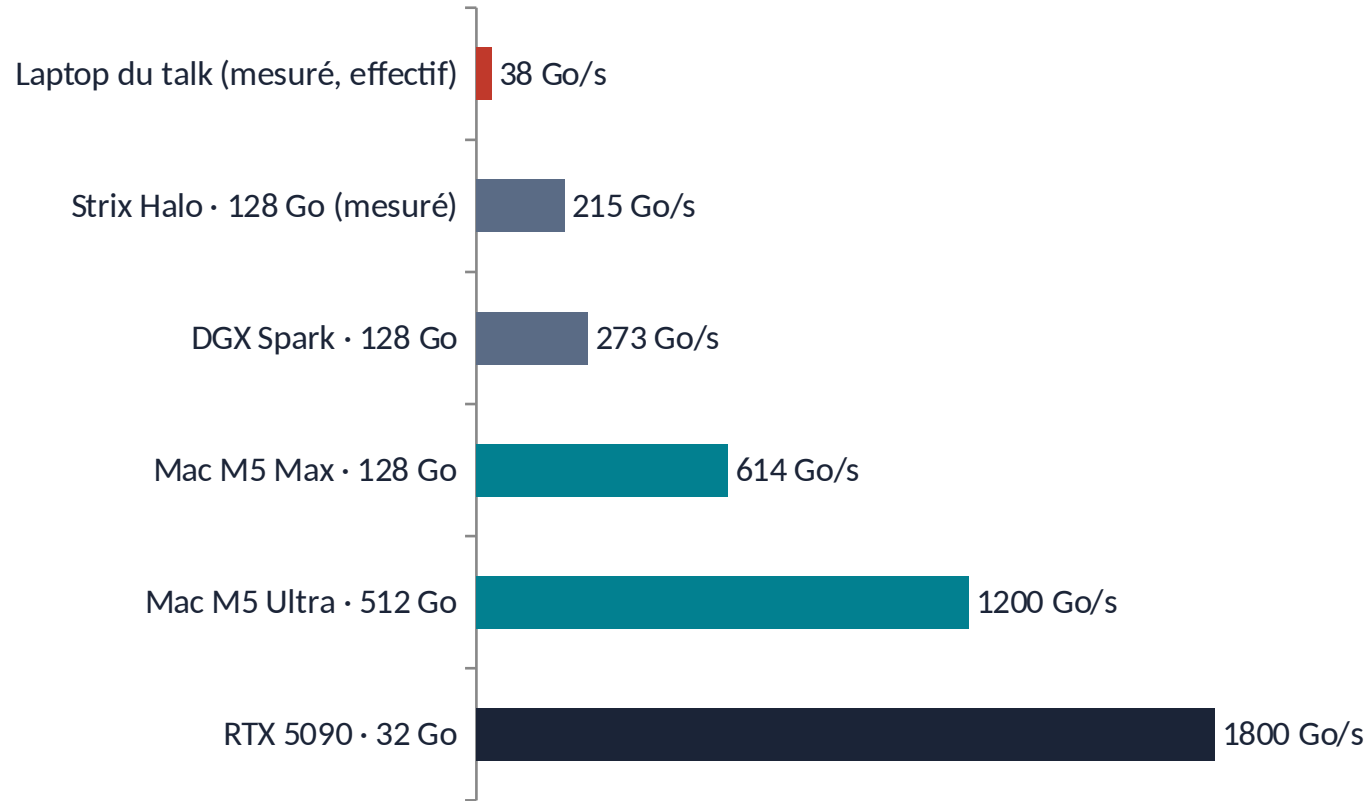
Attention hybride, contexte de 256 k à 1 M, modèles livrés quantifiés.

Les open-weights de tête pèsent 300 B à 2,8 T : pas sur un laptop.

Le modèle de la démo revendique 62,5 sur SWE-bench Pro : revendication de l'éditeur.

SWE-bench Verified est saturé ; **sur les tâches longues en terminal, l'écart reste réel.**

Matériel : lisez la bande passante, pas les TOPS



Lire le graphique

Rappel acte 1 : tok/s $\approx$ bande passante $\div$ poids actifs.

Repère officiel llama.cpp : gpt-oss-120b sur DGX Spark $\rightarrow$ 59 t/s à vide, 43 t/s à 32 k de contexte.

La RTX 5090 est la plus rapide, mais 32 Go seulement : les gros MoE n'y tiennent pas.

La DRAM est chère en 2026 (TrendForce : +10 à 18 % par trimestre).

Pour un agent local : **beaucoup de mémoire (≥ 128 Go) et de la bande passante, plutôt que de la puissance de calcul.**

Bilan confidentiel — l'inventaire de la journée :

RIEN.

Les prompts, le code de la démo, les tests, la skill, les correctifs de l'agent :
tout est resté sur les machines de cette salle.

Chez vous ce soir — trois commandes

```
$ git clone <le repo de cette présentation>
$ docker compose build
$ docker compose run --rm llama bash
# hf download <modèle> --local-dir /models/<modèle>
# llama-server -m /models/<modèle>/<fichier>.gguf
```

Le repo contient tout : conteneur llama.cpp, démo avec son bug, skill explain-file, config OpenCode, notes de recherche et mesures.

QR → repo public
github.com/<vous>/llama

(à générer avant le jour J)

Merci.

Place aux questions — et si le serveur tient, à une démo improvisée.

repo : github.com/<vous>/llama · licence MIT · tout est rejouable